Paravizor — Technical Specification

Version: 0.1.0-draft
Status: Pre-implementation
Last updated: 2026-03-30

1. Overview

Paravizor is a fully automated bug bounty reconnaissance pipeline written in Go. It takes in-scope and out-of-scope domains/URLs as input and runs a configurable DAG-based pipeline covering subdomain enumeration, fingerprinting, URL extraction, attack surface triage, static file extraction, and vulnerability scanning.

Core Principles

- **Declarative**: Pipelines and tools are defined in YAML. No recompilation needed to add tools or modify workflows.
- **Extensible**: Community-shareable YAML configs. Go plugin interface deferred to a later version.
- **Resilient**: Event-sourced checkpoint system. Crash recovery with zero data corruption and minimal redundant processing.
- **Decoupled**: Frontend (TUI/CLI) and backend communicate exclusively through an event bus. The UI layer is swappable.
- **Scope-first**: Out-of-scope filtering is middleware on every node transition. Poisoned data is caught early.

Target Scale

Single small-to-medium programs: hundreds of subdomains, thousands of URLs. The architecture does not target internet-scale enumeration.

2. Architecture Decisions

Decision	Choice	Rationale
Language	Go	Single binary, strong concurrency, mature ecosystem for CLI/TUI
Database	SQLite (WAL mode)	Zero-config, single-file per project, good enough for target scale
SQLite driver	`modernc.org/sqlite`	Pure Go, no CGO dependency, cross-compilation friendly
SQL Query Gen	sqlc	Type safe go code generation from sql queries
TUI framework	Bubbletea + Lipgloss + Bubbles	Elm architecture, composable, active community
CLI framework	Cobra + Viper	Industry standard, config binding, shell completion
Pipeline definition	YAML from day one	Declarative, readable, diffable, shareable
Tool definitions	YAML, declarative	No per-tool Go code. Generic runner handles all tools
Built-in tool defs	Embedded in binary (`go:embed`)	Single binary distribution. Users override via config files
Item types	Fixed set in Go	`domain`, `url`, `ip`, `port`, `dns_record`, `finding`, `file`
Plugin system	YAML pipeline/tool configs	Go plugin interface deferred to future version
Tool management	Check and warn, skip missing	Pipeline continues with available tools. Missing tools logged as warnings
Rate limiting	Per-target credit system	Dependency-weighted, dynamically reallocated. See section 7
AI integration	Semi-autonomous	Actions require user consent. Extension, not core feature

Decision	Choice	Rationale
Frontend-backend	Event bus pattern	Full decoupling. TUI, CLI, future web UI all subscribe to same events
Checkpoint/recovery	Event-sourced pipeline state	`pipeline_state` table in SQLite. Re-queue interrupted items on restart
Scope filtering	Middleware on every node transition	Items checked against scope rules before entering any node

3. Project Structure

```
paravizor/
├── cmd
│   └── paravizor
│       └── main.go # main entry point
├── docs # documentation
├── internal
│   ├── ai # ai stuff and integration
│   ├── assets
│   │   ├── assets.go # //go:embed pipelines/*.yaml tools/*.yaml
│   │   ├── pipelines # default yaml pipeline
│   │   └── tools # default yaml tools
│   ├── bootstrap
│   │   └── bootstrap.go # loads themes, tools, pipelines and validates on startup
│   ├── cli # cli commands/utils
│   ├── config # config loading, merging, handling and definition
│   ├── engine # main brains for orchestrating the work
│   ├── events # communicates events to the ui via a pubsub event bus
│   ├── export # export recon results to a report and other formats
│   ├── items # wrappers and definitions for process items (urls, domains)
│   ├── project # loading, saving and opening a project
│   ├── ratelimit # adaptive rate limit handling and credit pool
│   ├── runner # bootstrapping and starting a pipeline run (seeding db, making logs dir, disabling
missing tools...)
│   ├── store # models and queries
│   │   ├── db # automatically generated by sqlc
│   │   ├── migrations.go
│   │   ├── queries.go
│   │   ├── queries.sql
│   │   ├── schema.sql
│   │   ├── store.go
│   │   └── types.go
│   ├── theme # dynamic theme definition and loading
│   ├── tool # tool validation and handling
│   ├── tui # bubbletea terminal ui
│   └── utils # yaml parsing, validation and other shared utilities
```

4. Item Types

Items are the units of data that flow through the pipeline. They are a **fixed set** defined in Go, mapping directly to the database model.

> All items should have a `source` field to indicate which node/tool produced it.

Type	Description	Key Fields
`domain`	A (sub)domain name	`name`, `` `is_live ``, `ip`,
`url`	A full URL	`full_url`, `domain_id`, `path`, `query_string`, `status_code`, `content_type`,
`ip`	An IP address	`address`
`port`	An open port on an IP	`ip_id`, `port`, `service`,
`dns_record`	A DNS record	`domain_id`, `name`, `record_type`, `value`
`finding`	A vulnerability or notable discovery	`target_id`, `severity`, `title`, `description`, `evidence`
`file`	A downloaded static file	`url_id`, `file_path`, `file_type`, `size_bytes`

All item types implement a common interface:

```
type Item interface {
    Type() ItemType         // domain, url, ip, port, dns_record, finding, file
    ID() string             // unique identifier
    Source() string         // originating tool/node
    Scope() ScopeTarget     // returns data needed for scope checking
}
```

Items are serialized to/from the database and passed through pipeline channels as `Item` interfaces. Nodes declare what types they `consume` and `produce`.

5. Pipeline Engine

5.1 DAG Model

The pipeline is a Directed Acyclic Graph (DAG) of **nodes**. Each node wraps one or more tools and defines:

- **Input type(s)**: What item types it consumes
- **Output type(s)**: What item types it produces
- **Routes**: Conditional edges to downstream nodes
- **Batch policy**: How items are accumulated before processing
- **Stage**: Logical grouping (for visualization and ordering)

```yaml
# Example pipline config, global or per-project

pipeline:
  name: default-recon
  description: "Full automated recon pipeline"

  stages:
    - id: 0
      name: "Entry Point"
    - id: 1
      name: "Subdomain Enumeration"
    - id: 2
      name: "Fingerprinting"

  nodes:
```

```yaml
    # -- Stage 0: Entry Point -- Since each node can only have one tool
    # we use this as a fanout node to route to multiple nodes in parallel.
    # It's also possible to have one tool call multiple tools internally (subfinder, amass in parallel for
example)
    # but this is more flexible and easier to manage.

    - id: entry-point
      name: "Subdomain Enumeration Fanout Entry Point"
      stage: 0
      consumes: domain
      produces: domain
      routes:
        - to: passive-enum
        - to: active-bruteforce
        - to: permutation-expansion
        - to: live-filter

    # -- Stage 1: Subdomain Enumeration --
    - id: subfinder-passive-enum
      name: "Passive Enumeration"
      stage: 1
      tool: subfinder # these exist as "tool.yaml" files in the tools directory
      consumes: domain
      produces: domain
      routes:
        - to: sync-passive-enum

    - id: amass-passive-enum
      name: "Amass Passive Enumeration"
      stage: 1
      tool: amass-passive
      consumes: domain
      produces: domain
      routes:
        - to: sync-passive-enum

    - id: chaos-passive-enum
      name: "Chaos API Passive Enumeration"
      stage: 1
      tool: chaos-api
      consumes: domain
      produces: domain
      routes:
        - to: sync-passive-enum

    - id: sync-passive-enum
      name: "Assembly Point for syncing passive enumeration results"
      stage: 1
      consumes: domain
      produces: domain
      batch: # makes it wait for upstream nodes to finish to process everything in one go.
        wait_for_peers: true
      routes: # fanout to multiple nodes, can also have conditions for conditional routing
        - to: active-bruteforce
        - to: live-filter
        - to: subdomain-takeover

    - id: active-bruteforce
      name: "Active Bruteforce"
      stage: 1
      tool: puredns-bruteforce
      consumes: domain
      produces: domain
      routes:
```

```yaml
      - to: permutation-expansion
      - to: live-filter
      - to: subdomain-takeover

  - id: permutation-expansion
    name: "Permutation Expansion"
    stage: 1
    tool: alterx-puredns
    consumes: domain
    produces: domain
    batch:
      size: 100
      timeout: 30
    routes:
      - to: live-filter
      - to: subdomain-takeover

  - id: live-filter
    name: "Live Host Filter"
    stage: 1
    tool: dnsx-live
    consumes: domain
    produces: domain
    batch:
      size: 500
      timeout: 30
    routes:
      - to: screenshotting
      - to: port-scan
      - to: techstack
      - to: active-crawl
      - to: passive-crawl
      - to: nuclei-host

  # ── Stage 2: Fingerprinting ──
  - id: screenshotting
    name: "Screenshotting"
    stage: 2
    tool: haylxon
    consumes: domain
    produces: file
    batch:
      size: 50
      timeout: 15
    # terminal node (no routes)

  - id: port-scan
    name: "Port Scanning"
    stage: 2
    tool: dnsx-rustscan
    consumes: domain
    produces: port
    batch:
      size: 50
      timeout: 30
    # terminal node

  - id: techstack
    name: "Techstack Fingerprinting Fanout"
    stage: 2
    consumes: domain
    produces: domain
    routes:
      - to: techstack-httpx
```

```yaml
      - to: techstack-nuclei
      - to: techstack-whatcms

    - id: techstack-httpx
      name: "httpx Techstack Fingerprinting"
      stage: 2
      tool: httpx-tech
      consumes: domain
      produces: techstack
      batch:
        size: 50
        timeout: 30
      routes:
        - to: techstack-sync
    - id: techstack-nuclei
      name: "Nuclei Techstack Fingerprinting"
      stage: 2
      tool: nuclei-tech
      consumes: domain
      produces: techstack
      batch:
        size: 50
        timeout: 30
      routes:
        - to: techstack-sync

    - id: techstack-whatcms
      name: "WhatCMS Techstack Fingerprinting"
      stage: 2
      tool: whatcms
      consumes: domain
      produces: techstack
      batch:
        size: 50
        timeout: 30
      routes:
        - to: techstack-sync

    - id: techstack-sync
      name: "Assembly Point for syncing techstack results"
      stage: 2
      consumes: techstack
      produces: techstack
      batch:
        wait_for_peers: true
      routes:
        - to: wordpress-theme
          condition: "'wordpress' in techstack"
        - to: wpscan
          condition: "'wordpress' in techstack"

    - id: wordpress-theme
      name: "WordPress Theme Detection"
      stage: 2
      tool: themedetect
      consumes: domain
      produces: domain
      batch:
        size: 10
        timeout: 10s
      # terminal node
```

5.2 Node Interface

```go
type Node interface {
    ID() string
    Name() string
    Stage() int
    Consumes() []ItemType
    Produces() []ItemType

    // Process a batch of items, return results
    Process(ctx context.Context, items []Item) ([]Item, error)

    // Lifecycle
    Start(ctx context.Context) error
    Stop() error

    // Status
    Status() NodeStatus       // idle, active, draining, completed, error
    Stats() NodeStats         // items_in, items_out, errors, duration
}
```

5.3 Execution Flow

1. **DAG construction**: Pipeline YAML is parsed into a DAG structure. Topological sort validates no cycles. Dependency weights are computed (section 7).
2. **Channel wiring**: Each edge in the DAG becomes a buffered Go channel. The buffer size is configurable (default: 10000 items). Multiple edges from the same source use fan-out (each item is sent to all matching routes).
3. **Scheduler loop**: The engine runs a scheduler goroutine per node that:
   - Reads from the node's input channel(s)
   - Accumulates items according to the batch policy
   - Applies scope filtering (middleware)
   - Sends the batch to the node's `Process` method
   - Routes output items to downstream channels
   - Persists state transitions to `pipeline_state`
4. **Database-backed data flow**: Node outputs are persisted to SQLite immediately. Downstream nodes query the database for pending items when they're ready to process. There is no channel-based back-pressure, the database acts as an unbounded, persistent buffer. If an upstream node finishes fast, its data is safely stored and available for downstream nodes to fetch on their own schedule.
5. **Completion propagation**: When a node has processed all items and all its upstream nodes are `completed`, it transitions to `draining` (finishing current batch) then `completed`. This signal propagates forward through the DAG. Terminal nodes (no outgoing edges) complete when their input is fully processed.

5.4 Batch Policy

Batching controls how items are accumulated before being sent to a node for processing:

```yaml
batch:
  size: 500         # min items to justify processing (avoids tiny batches)
  timeout: 30s      # max wait time before processing a partial batch (less than size)
  wait_for_peers: true  # wait for other upstream nodes before processing
```

The `wait_for_peers` flag is critical for assembly points. Example: the "Deduplication" node receives URLs from both `katana` (active crawl) and `gau` / `waybackurls` (passive crawl). Setting `wait_for_peers: true` makes it wait until all upstream nodes are `draining` or `completed` before processing, ensuring maximum deduplication efficiency.

Without `wait_for_peers`, a dedup node might process a batch from katana, then later receive duplicates from gau that it can't catch.

5.5 Conditional Routing

Routes can have conditions evaluated against the output item's fields:

```
routes:
  - to: wordpress-theme
    condition: "'wordpress' in techstack"
  - to: port-scan
    # no condition = unconditional, all items routed here
```

Conditions are simple expressions evaluated by a built-in evaluator (not a full scripting language). Supported operators:

- `'value' in field` — field (string or list) contains value
- `field == 'value'` — exact match
- `field != 'value'` — not equal
- `field > N` / `field < N` — numeric comparison
- `field matches 'regex'` — regex match
- `and`, `or`, `not` — logical operators

5.6 Parallel, Sequential, and Chain Nodes

It was proposed as a feature to add additional logic to nodes themselves on how they would pass around data from one tool to another internally, but this added a lot of additional complexity. The current architechture is to limit one tool per node (that tool can still have multiple internal commands withing it, even though it's discouraged), and the logic would be handled by the internode routing (see above).

6. Tool Definition System

6.1 Design Philosophy

Zero Go code per tool. Adding support for a new tool means writing a YAML definition file. The Go codebase provides a generic tool runner that handles process execution, I/O, output parsing, and lifecycle management for all tools uniformly.

6.2 Tool Definition Schema

```
tool:
  name: subfinder                    # unique identifier, referenced by pipeline nodes
  binary: "/usr/bin/subfinder"                  # executable name or full path
  description: "Passive subdomain enumeration"
  version_flag: "-version"  # command to check version and confirm existence

  install:                           # install hints (displayed when tool is missing)
    - "go get github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest"

  input:
    type: arg                        # arg | stdin | file | none
    flag: "-d"                       # flag prefix for arg type
    bulk:                            # bulk input mode (overrides single input)
      type: file                     # write items to tempfile, pass path
      flag: "-dL"
      separator: "\n"

  output:
```

```yaml
    type: stdout                      # stdout | file | directory
    format: line                      # line | json | jsonl | csv | regex | xml
    # For non-stdout:
    # path: "{{.TempDir}}/output.txt"
    # flag: "-o"

    # Field mapping (for structured formats):
    fields:                           # maps parsed output to item fields
      name: "$.host"                  # JSON path (for json/jsonl)
      # or for regex: name: "{{.domain}}"

  flags:                              # default flags, always applied
    - "-silent"
    - "-all"

  env: {}                             # environment variables to set

  rate_limit:
    flag: "-rl"                       # flag that controls the tool's internal rate limit
    unit: "s"                       # what the flag value means (s/m/ms...)

  timeout:
    flag: "-timeout"
    default: 300                      # seconds, applied if no flag override. '0'means no timeout by
default

  consumes: domain                    # item type consumed
  produces: domain                    # item type produced
```

6.3 Output Parsing

The generic runner includes built-in parsers for common output formats:

Format	How It Works	Example Tool
`line`	Split stdout by newline. Each line = one item.	subfinder, puredns
`json`	Parse entire stdout as JSON object/array. Extract fields via JSON path.	httpx `-json`
`jsonl`	Parse each stdout line as a separate JSON object.	nuclei `-jsonl`
`csv`	Parse as CSV with column headers. Map columns to fields.	Custom tools
`regex`	Apply named capture groups to each line.	rustscan output
`xml`	Parse as XML. Extract via XPath expressions.	nmap `-oX`

Field mapping examples:

```yaml
# JSON (httpx)
output:
  format: jsonl
  fields:
    domain: "$.host"
    status_code: "$.status_code"
    content_type: "$.content_type"
    tech: "$.tech"                    # array field

# Regex (rustscan)
output:
  format: regex
```

```
  pattern: '^(?P<ip>[\d.]+) -> \[(?P<ports>[\d,]+)\]'
  fields:
    ip: "{{.ip}}"
    port: "{{.ports}}"              # comma-separated, split into multiple items

# Line (subfinder)
output:
  format: line
  fields:
    name: "{{.line}}"               # whole line is the domain name
```

6.4 Tool Registry

At startup, the tool registry:

1. Loads embedded tool definitions (from `go:embed`)
2. Loads global user tool definitions from `~/.config/paravizor/tools/*.yaml`
3. Loads project-level tool definitions from `./tools/*.yaml`
4. Merges: project overrides global overrides embedded
5. Checks binary availability for each tool (`which <binary>` or configured path)
6. Logs warnings for missing tools, marks them as unavailable

When a pipeline node references a tool, the registry provides the full merged definition. If the tool is unavailable, the node is skipped with a warning (pipeline continues).

7. Rate Limit Credit System

7.1 Overview

The rate limiter implements a **per-target credit pool** that distributes request capacity across active scanner nodes. The system is designed to:

- Prevent overwhelming targets and triggering WAF/rate-limiting
- Maximize throughput for high-priority tools that unblock more of the pipeline
- Dynamically reallocate capacity as nodes start and finish
- React to target feedback (429 responses, timeouts)

7.2 Core Concepts

Credit Pool

A global per-target budget expressed in requests/second (e.g., 50 req/s). This is the total outbound request rate across all active tools for a given target.

Credits (Tokens)

The budget is divided into discrete tokens. Each active node is allocated a portion of the total tokens. A node's tool runner can only dispatch requests at its allocated rate.

Rate Control Layer

Rate control operates at the **Go tool runner level**, not via tool CLI flags. The runner wraps the tool's input feed and throttles how fast items are sent to the external process. This is critical because:

- Not all tools have rate-limit flags
- Dynamic adjustment requires changing the rate mid-execution

- The runner's token bucket can be updated in real-time without restarting the tool process

Tool-native rate limit flags (like nuclei's `-rl`) are set as a **secondary safety net** at the initial allocated rate, but the primary control is always the runner's input throttle.

7.3 Dependency-Weighted Allocation

Tokens are allocated proportionally to a node's **downstream dependency weight**. The weight represents how many nodes in the DAG transitively depend on this node's output.

Weight Computation

At pipeline startup, the engine computes weights from the DAG:

```
weight(node) = 1 + sum(weight(child) for child in node.direct_dependents)
```

Terminal nodes (no outgoing edges) have `weight = 1`. Nodes with many transitive dependents have higher weights.

Example from the default pipeline:

Node	Downstream Dependents	Weight
`live-filter` (dnsx)	screenshotting, port-scan, techstack, active-crawl, passive-crawl, nuclei-host, + their transitive deps	~25
`httpx-liveliness`	static-file-extraction (4 sub-nodes), parametrized-url-extraction, + their deps	~8
`techstack` (httpx/nuclei/whatcms)	wordpress-theme, wpscan	3
`nuclei-url-scan`	(terminal)	1
`screenshotting`	(terminal)	1

Allocation Formula

```
node_tokens = floor((node_weight / sum_of_active_weights) * total_budget)
```

If `total_budget = 50 req/s` and two nodes are active with weights 25 and 5:

- Node A: `floor((25/30) * 50) = 41 req/s`
- Node B: `floor((5/30) * 50) = 8 req/s`
- Remaining 1 req/s goes to the burst reserve (see 7.5)

7.4 Node Completion & Dynamic Reallocation

Completion Signaling

Nodes progress through these states:

```
idle → active → draining → completed
                          ↗
                error ──┘
```

- **idle**: Node exists in the DAG but has received no items yet.
- **active**: Node is consuming items and producing output. It holds an allocation from the credit pool.
- **draining**: All upstream nodes that route to this node are `completed` AND the node's input channel is empty. The node is

finishing its last batch. It signals the credit pool: "I will release my tokens soon." Other active nodes can start planning to absorb these tokens.
- **completed**: Node has finished all processing. Its tokens are **immediately released** back to the pool.
- **error**: Node encountered a fatal error. Tokens released.

How a Node Knows It's Done

The pipeline engine maintains a **pending upstream count** per node: the number of upstream nodes that haven't yet `completed`. When this count reaches zero AND the input channel is drained, the node transitions to `draining`.

This is event-driven: when any node completes, it notifies the engine, which decrements the pending count for all direct downstream nodes and checks if any can transition.

Reallocation Trigger

When a node releases tokens (transitions to `completed` or `error`), the credit pool triggers a **rebalance cycle**:

1. Collect all currently `active` nodes and their weights.
2. Recalculate allocations: `node_tokens = (node_weight / sum_active_weights) * total_budget`
3. For each active node, update its token bucket rate. This takes effect immediately. the running tool process does not restart, only the runner's throttle rate changes.
4. Emit a `RateLimitRebalanced` event for observability.

Example flow:

```
T=0:  active=[A(w=25), B(w=5), C(w=3)]  budget=50
      A=37, B=7, C=4, burst_reserve=2

T=30: C completes. Releases 4 tokens. Rebalance:
      active=[A(w=25), B(w=5)]  budget=50
      A=41, B=8, burst_reserve=1
      → A's runner rate updates from 37→41 (immediate, no restart)
      → B's runner rate updates from 7→8 (immediate, no restart)

T=60: A completes. Releases 41 tokens. Rebalance:
      active=[B(w=5)]  budget=50
      B=50
      → B gets the entire budget
```

7.5 Burst Allowance

When a new scanner node activates (transitions from `idle` → `active`) and all tokens are currently allocated, the system handles this gracefully:

Burst Reserve

A small portion (default: 10% of total budget, minimum 5 req/s) is held as **burst reserve**. This reserve is always available for newly activating nodes so they can start immediately without waiting for a full rebalance.

Activation Flow

1. Node transitions to `active`.
2. Credit pool checks if unallocated tokens exist.
   - **If yes**: Assign from unallocated pool. Trigger proportional rebalance.
   - **If no unallocated tokens**: Assign from burst reserve (up to reserve limit).
3. Trigger full rebalance: all active nodes' allocations are recalculated based on current weights.
4. The burst reserve is replenished from any rounding remainders.

Temporary Over-Budget

During the brief window between a node activating from burst reserve and the rebalance completing, the total active allocation may exceed the base budget by up to the burst reserve amount. This is acceptable because:

- The window is milliseconds (rebalance is a fast in-memory operation)
- The burst reserve is small (5-10% of budget)
- It prevents the alternative: a node sitting idle waiting for tokens while work is available

7.6 Adaptive Throttling

The credit pool monitors tool runner feedback for signs the target is rate-limiting:

Signal	Response
HTTP 429 (Too Many Requests)	Reduce offending node's rate by 50%. Wait `Retry-After` if present.
Consecutive timeouts (>3)	Reduce rate by 25%
Connection resets	Reduce rate by 25%
Successful responses resume	Gradually increase rate back to allocated level (10% per minute)

Adaptive throttling operates **per-node** and adjusts the node's effective rate below its allocated ceiling. The freed capacity is NOT redistributed immediately, it's held in reserve for when the throttled node recovers or after a set timeout (5 minutes by default).

If adaptive throttling reduces a node's effective rate, the `RateThrottled` event is emitted.

7.7 Overdrive Mode

Overdrive mode bypasses the entire credit system. All tool runners dispatch items as fast as the tool can consume them. Activated per-project via:

```
# project config
rate_limit:
  mode: overdrive   # normal | overdrive
```

Or toggled at runtime via TUI/CLI command. A warning is displayed: rate limiting is disabled, target may block you.

7.8 Configuration

```
# In project.yaml config
rate_limit:
  mode: normal                    # normal | overdrive
  budget: 50                      # total req/s per target
  burst_reserve_percent: 10       # percentage held for burst
  burst_reserve_min: 5            # minimum burst reserve (req/s)
  adaptive:
    enabled: true
    backoff_factor: 0.5           # rate multiplier on 429
    recovery_rate: 0.1            # rate increase per minute after recovery
    max_consecutive_timeouts: 3   # timeouts before throttling
    redistribution_timeout: 600   # seconds after which held capacity is added back to the global pool

  # Per-tool overrides (optional, caps the tool regardless of pool allocation)
  tool_overrides:
    nuclei:
      max_rate: 30                # nuclei never gets more than 30 req/s
    katana:
```

```
      max_rate: 20
```

8. Scope Engine

8.1 Purpose

The scope engine is the first line of defense against processing out-of-scope items. It is applied as **middleware on every node transition**: before an item enters any node's input channel, it is checked against the scope rules. Out-of-scope items are dropped and logged.

8.2 Scope Rules

The scope engine should automatically match and classify entries as wildcard or exact entries for domains, urls, regex, endpoints...

```yaml
# Project scope configuration, in project.yaml
scope:
  include:
    - "*.target.com"
    - "target.com"
    - "https://target.com/app/*"

  exclude:
    - "sso.target.com"
    - "target.com/affiliates/*"
    - "*.cdn.target.com"
    - "^staging-\\d+\\.target\\.com$"
```

8.3 Matching Logic

1. Rules are sorted by generality (exact > wildcard), then by specificity (out-of-scope > in-scope for same priority).
2. For each item, the engine checks rules in order. First match wins.
3. If no rule matches, the item is **out of scope** (default deny).

The single entry point:

```go
func (s *ScopeEngine) IsInScope(item Item) bool
```

This function dispatches to type-specific matchers:

- **Domain items**: Checked against wildcard/exact domain patterns
- **URL items**: Checked against path patterns AND the domain portion against domain patterns
- **IP items**: Checked against domain patterns (if defined)

8.4 Pipeline Integration

Scope checking is implemented as middleware in the router, not in individual nodes:

```go
// In router.go, before sending item to downstream channel:
if !scopeEngine.IsInScope(item) {
    eventBus.Publish(OutOfScopeFiltered{Item: item, Node: sourceNode})
    store.DeleteItem(item) // remove from DB
    continue
}
downstream <- item
```

Items that are filtered out are:

1. Removed from the database (not just flagged, the spec says "completely erase")
2. Logged via event bus for visibility
3. Counted in node stats (`items_filtered`)

8.5 Early Filtering

Per the spec: filtering should happen as early as possible. The scope middleware runs at every node transition, but tools that support scope-aware flags should use them:

```
# In tool definition
tool:
  name: katana
  scope_flags:
    exclude: "-exclude-pattern"     # pass out-of-scope patterns
    include: "-scope-pattern"       # pass in-scope patterns
```

When present, the tool runner injects scope patterns into the tool's command line, providing defense-in-depth (tool-level + router-level filtering).

9. Data Model (SQLite)

9.1 Overview

Each project has its own SQLite database file in WAL mode. The schema covers pipeline items, process tracking, pipeline state, and metadata.

9.2 Schema

```
-- Schema for paravizor project database.
-- Table order matters: referenced tables must come before referencing ones.

CREATE TABLE IF NOT EXISTS scope_rules (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    pattern    TEXT    NOT NULL,
    wildcard   BOOLEAN NOT NULL DEFAULT 0, -- 0 for exact, 1 for wildcard/regex
    type       TEXT    NOT NULL CHECK(type IN ('path','domain','ip')),
    in_scope   BOOLEAN NOT NULL DEFAULT 1, -- 1 for in-scope, 0 for out-of-scope
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS batches (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    node_id      TEXT    NOT NULL,
    item_count   INTEGER NOT NULL DEFAULT 0,
    status       TEXT    NOT NULL DEFAULT 'pending'
                 CHECK(status IN ('pending','processing','completed','failed')),
    created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    completed_at DATETIME
);
CREATE INDEX IF NOT EXISTS idx_batches_node   ON batches(node_id);
CREATE INDEX IF NOT EXISTS idx_batches_status ON batches(status);

CREATE TABLE IF NOT EXISTS domains (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    name       TEXT    NOT NULL UNIQUE,
    source     TEXT    NOT NULL,
```

```sql
    is_live    BOOLEAN,
    ip         TEXT,
    batch_id   INTEGER REFERENCES batches(id),
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_domains_name ON domains(name);
CREATE INDEX IF NOT EXISTS idx_domains_live ON domains(is_live);

CREATE TABLE IF NOT EXISTS techstack (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    domain_id  INTEGER NOT NULL REFERENCES domains(id) ON DELETE CASCADE,
    technology TEXT    NOT NULL,
    version    TEXT,
    category   TEXT,
    source     TEXT    NOT NULL,
    UNIQUE(domain_id, technology)
);
CREATE INDEX IF NOT EXISTS idx_techstack_domain ON techstack(domain_id);
CREATE INDEX IF NOT EXISTS idx_techstack_tech   ON techstack(technology);

CREATE TABLE IF NOT EXISTS ips (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    address    TEXT    NOT NULL UNIQUE,
    info       TEXT, -- additional info, like the associated domain/dns record
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS ports (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    ip_id    INTEGER NOT NULL REFERENCES ips(id) ON DELETE CASCADE,
    port     INTEGER NOT NULL,
    service  TEXT,
    source   TEXT    NOT NULL,
    UNIQUE(ip_id, port)
);
CREATE INDEX IF NOT EXISTS idx_ports_ip ON ports(ip_id);

CREATE TABLE IF NOT EXISTS urls (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    full_url     TEXT    NOT NULL UNIQUE,
    domain_id    INTEGER REFERENCES domains(id),
    path         TEXT,
    query_string TEXT,
    status_code  INTEGER,
    content_type TEXT,
    source       TEXT    NOT NULL,
    batch_id     INTEGER REFERENCES batches(id),
    created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_urls_domain ON urls(domain_id);
CREATE INDEX IF NOT EXISTS idx_urls_status ON urls(status_code);

CREATE TABLE IF NOT EXISTS url_flags (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    url_id     INTEGER NOT NULL REFERENCES urls(id) ON DELETE CASCADE,
    flag_type  TEXT    NOT NULL,
    flag_value TEXT,
    source     TEXT    NOT NULL,
    UNIQUE(url_id, flag_type, flag_value)
);
CREATE INDEX IF NOT EXISTS idx_url_flags_url  ON url_flags(url_id);
CREATE INDEX IF NOT EXISTS idx_url_flags_type ON url_flags(flag_type);
```

```sql
CREATE TABLE IF NOT EXISTS findings (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    -- could be either one
    domain_id   INTEGER REFERENCES domains(id),
    url_id      INTEGER REFERENCES urls(id),
    severity    TEXT    CHECK(severity IN ('info','low','medium','high','critical')),
    title       TEXT    NOT NULL,
    description TEXT,
    evidence    TEXT,
    source     TEXT    NOT NULL,
    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_findings_severity ON findings(severity);
CREATE INDEX IF NOT EXISTS idx_findings_scanner  ON findings(source);
CREATE INDEX IF NOT EXISTS idx_findings_domain   ON findings(domain_id);

CREATE TABLE IF NOT EXISTS processes (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    name           TEXT    NOT NULL,
    command        TEXT    NOT NULL,
    pid            INTEGER,
    node_id        TEXT    NOT NULL,
    batch_id       INTEGER REFERENCES batches(id),
    status         TEXT    NOT NULL DEFAULT 'running'
                   CHECK(status IN ('running','completed','failed','killed','timeout')),
    exit_code      INTEGER,
    stdout_path    TEXT,
    stderr_path    TEXT,
    started_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    completed_at   DATETIME,
    last_heartbeat DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_processes_status ON processes(status);
CREATE INDEX IF NOT EXISTS idx_processes_node   ON processes(node_id);

CREATE TABLE IF NOT EXISTS pipeline_state (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    item_type    TEXT    NOT NULL,
    item_id      INTEGER NOT NULL,
    node_id      TEXT    NOT NULL,
    status       TEXT    NOT NULL DEFAULT 'pending'
                 CHECK(status IN ('pending','processing','completed','failed','skipped')),
    started_at   DATETIME,
    completed_at DATETIME,
    error        TEXT,
    UNIQUE(item_id, item_type, node_id)
);
CREATE INDEX IF NOT EXISTS idx_pipeline_state_node   ON pipeline_state(node_id);
CREATE INDEX IF NOT EXISTS idx_pipeline_state_status ON pipeline_state(status);

CREATE TABLE IF NOT EXISTS dns_records (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    domain_id   INTEGER REFERENCES domains(id) ON DELETE CASCADE,
    record_type TEXT    NOT NULL,
    value       TEXT    NOT NULL,
    name        TEXT, -- if the dns record is not a, A or CNAME, very unlikely thhough, could be omited
    source      TEXT    NOT NULL,
    UNIQUE(domain_id, record_type, value)
);
CREATE INDEX IF NOT EXISTS idx_dns_records_domain ON dns_records(domain_id);

CREATE TABLE IF NOT EXISTS downloaded_files (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
```

```sql
    url_id     INTEGER NOT NULL REFERENCES urls(id) ON DELETE CASCADE,
    file_path  TEXT    NOT NULL,
    file_type  TEXT    NOT NULL,
    size_bytes INTEGER,
    sha256     TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_downloaded_files_url  ON downloaded_files(url_id);
CREATE INDEX IF NOT EXISTS idx_downloaded_files_type ON downloaded_files(file_type);

-- user created notes throughout the project
CREATE TABLE IF NOT EXISTS notes (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    content    TEXT    NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

9.3 Concurrency

SQLite in WAL mode supports concurrent reads with a single writer. The store layer serializes writes through a single goroutine with a write queue (buffered channel). Reads are concurrent via a connection pool.

Batch inserts use transactions with prepared statements for throughput. The target is >10,000 inserts/second, which WAL-mode SQLite handles comfortably.

9.4 Data Traceability

Every item has a `source` field tracing it to the tool that discovered it. The `batch_id` traces it to the pipeline execution batch. The `pipeline_state` table provides full provenance: which nodes processed this item, when, and with what result.

9.5 Global DB

In addition to the per project `paravizor.db` , a global databse will also be implemnted to keep stats and tabs about the user's utilisation of the tools. Here are the proposed tables:

- Project: project name, path, status (completed, paused...)
- Tools: tool name, checksum (to verify if a tool.yaml changed between runs), is_installed
- Pipeline: name, checksum (to verify if a pipeline.yaml changed between runs), integrity (enum:valid,broken,incomplete)

Other suggestions are appreciated

10. Event Bus

10.1 Purpose

The event bus is the sole communication channel between the backend (pipeline engine, store, tool runners) and the frontend (TUI, CLI). This decoupling means the UI layer is fully swappable.

10.2 Event Types

```go
// Core pipeline events
type NodeStarted struct {
    NodeID string
    Time   time.Time
}

type NodeCompleted struct {
    NodeID    string
    ItemsIn   int
```

```go
    ItemsOut  int
    Duration  time.Duration
}

type NodeError struct {
    NodeID  string
    Error   error
    Fatal   bool        // if true, node has stopped
}

type BatchStarted struct {
    BatchID  int64
    NodeID   string
    ItemCount int
}

type BatchCompleted struct {
    BatchID   int64
    NodeID    string
    ItemCount int
    Duration  time.Duration
}

// Item discovery events
// It is recommended to not process most of the discovery or filtering events
// to not overwhelm the ui
type DomainDiscovered struct {
    Domain  Domain
    Source  string
    NodeID  string
}

type URLDiscovered struct {
    URL    URL
    Source string
    NodeID string
}

type FindingDiscovered struct {
    Finding Finding
    NodeID  string
}

// Scope events
type OutOfScopeFiltered struct {
    ItemType string
    ItemID   string
    NodeID   string
    Reason   string
}

// Process events
type ProcessStarted struct {
    ProcessID int64
    ToolName  string
    Command   string
    PID       int
}

type ProcessCompleted struct {
    ProcessID int64
    ToolName  string
    ExitCode  int
    Duration  time.Duration
```

```go
}

type ProcessOutput struct {
    ProcessID int64
    Stream    string    // "stdout" | "stderr"
    Line      string
}

// Rate limit events
type RateLimitRebalanced struct {
    Allocations map[string]float64    // node_id -> req/s
    TotalBudget float64
}

type RateThrottled struct {
    NodeID    string
    OldRate   float64
    NewRate   float64
    Reason    string   // "429", "timeout", "reset"
}

// Pipeline lifecycle events
type PipelineStarted struct {
    PipelineID string
    NodeCount  int
}

type PipelineCompleted struct {
    Duration    time.Duration
    TotalItems  int
    TotalErrors int
}

type CheckpointSaved struct {
    Time time.Time
}

//other events are to be defined
```

10.3 Subscription Model

```go
type EventBus interface {
    Publish(event interface{})
    Subscribe(eventType reflect.Type, handler func(interface{}))
    SubscribeAll(handler func(interface{}))
}
```

- **TUI**: Subscribes via bubbletea's `Msg` system. The TUI adapter converts events into bubbletea messages sent to the `Update` function.
- **CLI**: Subscribes via a log formatter that renders events as structured log lines to stdout.
- **Future web UI**: Would subscribe via WebSocket, converting events to JSON messages.

10.4 Event Ordering

Events are published in order from a single goroutine per event source. The bus maintains a global sequence number for ordering across sources. Subscribers receive events in sequence order.

11. Configuration Hierarchy

11.1 Layers

Configuration is loaded and merged in this order (later overrides earlier):

```
1. Built-in defaults (embedded in binary)
   └── 2. Global user config (~/.config/paravizor/)
        ├── config.yaml          (API keys, tool paths, global rate limits)
        ├── tools/*.yaml         (user tool definitions/overrides)
        └── pipelines/*.yaml     (user pipeline definitions)
             └── 3. Project config (./project-dir/)
                  └── project.yaml   (project pipeline, scope, and rate limit)
```

11.2 Global Config

```yaml
# Example global

# ~/.config/paravizor/config.yaml
paravizor:
  # API keys
  api_keys:
    chaos: "your-key-here" # key value store

  theme: "catpuccin-mocha" # looks inside ~/.config/paravizor/themes for the theme definition
  default_pipeline: default # looks inside ~/.config/paravizor/pipelines for the pipeline definition

  max_concurrent_processes: 10
  process_healthcheck_interval: 5

  db:
    busy_timeout: 5000
    mmap_size: 268435456
    cache_size: 8000


  log_level: "info"
  ai:
    enabled: false
    provider: ""
    model: ""
    base_url: ""
    api_key: ""
    consent_mode: "always_ask"
```

11.3 Project Config

```yaml
# Example project config

project:
  name: "Example Recon Project"
  description: "Bug bounty recon project for example.com"
  pipeline: "default-recon" # looks inside ~/.config/paravizor/pipelines for the pipeline definition
  created_at: 2026-03-30T12:00:00Z
  scope:
    include:
      - "example.com"
      - "*.example.com"
    exclude:
```

```yaml
      - "dev.example.com"

  rate_limit_mode: "normal"
  rate_limit:
    - nodes: # This needs more granular tuning, still figuring out the best way to do this
      - "node-name"
      budget: 50 # requests per second
      burst_reserve_percentage: 10
      burst_reserve_min: 5
```

11.4 Viper Integration

Cobra commands bind to Viper for config loading. Environment variables override config files with prefix `PARAVIZOR_`:

```
PARAVIZOR_API_KEYS_CHAOS=xxx
PARAVIZOR_RATE_LIMIT_BUDGET=100
PARAVIZOR_LOG_LEVEL=debug
```

12. Checkpoint & Recovery

12.1 State Tracking

The `pipeline_state` table records the processing status of every item at every node:

item_id	item_type	node_id	status	started_at	completed_at	error
42	domain	live-filter	completed	...	...	NULL
42	domain	techstack	processing	...	NULL	NULL
107	url	httpx-live	failed	...	...	"timeout"

State transitions are written synchronously within the node's processing transaction. This means if the process crashes mid-batch, the items will still show `status='processing'`.

12.2 Checkpoint Saves

Periodic checkpoint saves (default: every 30 seconds) persist:

- All `pipeline_state` entries
- Node status (idle/active/draining/completed)
- Credit pool allocations
- Batch states
- Process states

Checkpoints are transactional. the entire state is written atomically.

12.3 Recovery Flow

On startup, if a project's database contains `pipeline_state` entries with `status='processing'`:

1. **Detect interrupted session**: Check for `processing` entries in `pipeline_state`.
2. **Prompt user**: "Previous session was interrupted. Resume from checkpoint? [Y/n]"
3. **If resume**:
   a. Mark all `status='processing'` entries as `status='pending'` (re-queue).
   b. Rebuild the DAG from the pipeline YAML.
   c. For each node, query its pending items and feed them into the node's input channel.

d. Nodes that were `completed` in the previous session are skipped (all their items are `completed`).
e. Pipeline execution resumes.

4. **If not resume**: User can choose to start fresh (clears all pipeline_state, keeps discovered items) or archive the project.

12.4 Graceful Shutdown

On SIGINT/SIGTERM or user-initiated pause:

1. Stop feeding new items to nodes.
2. Wait for in-flight batches to complete (with timeout).
3. Kill any running external processes gracefully (SIGTERM, then SIGKILL after 10s).
4. Write final checkpoint.
5. Log summary: "Session paused. X items pending, Y items completed."

12.5 Data Integrity

- All database writes are transactional.
- External process outputs are written to log files on disk, not held in memory.
- The pipeline state and item data are always consistent: if an item is `completed` at a node, its data (enriched fields) is committed in the same transaction.

13. CLI

Initially, paravizor was supposed to be operable with pure cli commands, but that proved to be counterproductive and opposite to the entire philosophy of the app. So the cli will be more destined for passive interactivity with the tools, projects and database

13.1 Command Structure

```
paravizor
├── init <name> [--dir <path>]       # Create new project
├── query <sql> [--project <path/name>]      # Raw SQL query against project DB or opens interactive
sqlite command line (like opencode)
├── export [--project <path/name>]   # Export artifacts of a given project (default current dir)
│   ├── artifacts [--dir <path>]     # Export txt files
│   ├── obsidian [--output <path>]   # Export Obsidian canvas
│   └── report [--output <path>]     # Export markdown report
├── tools                            # Tool management
│   ├── list                         # List all known tools and availability
│   ├── check                        # Check which tools are installed
│   └── show <name>                  # Show tool definition details
├── project                          # Project management
│   ├── list                         # List all projects
│   ├── import <path>                # Import project from DB file
│   └── archive <name>               # Archive a project
└── version                          # Show version info
```

14. TUI

14.1 Architecture

The TUI is a Bubbletea application with a page-router pattern. The main model holds:

- Current page state

- Global status bar state
- Event log buffer
- Pipeline status cache (updated via event bus)

Events from the backend are converted to bubbletea `Msg` types and sent via `p.Send()`.

14.2 Pages

Project Page (Start Screen)

Shown on startup. Displays:

- Environment check results (installed tools, API keys configured)
- List of existing projects (resume option)
- Create new project form (name, directory, initial scope)
- Import project option

Dashboard

The main view during execution. Grid layout:

```
┌──────────────────────────────────────────────────┐
│ Paravizor - Target Program           [Running]   │
├──────────────────────┬───────────────────────────┤
│ Subdomains:    198   │ Live:         142         │
│ URLs:        1,847   │ Findings:      23         │
│ IPs:            67   │ Ports:        312         │
│ Files:          89   │ Errors:         2         │
├──────────────────────┴───────────────────────────┤
│ Recent Events                                    │
│ [12:04] nuclei found CVE-2024-1234 on app.tgt..  │
│ [12:03] httpx completed batch #7 (200 urls)      │
│ [12:03] katana discovered 847 urls on tgt.com    │
│ [12:02] subfinder found 142 subdomains           │
├──────────────────────────────────────────────────┤
│ Pipeline (simplified)                            │
│ [■] Enum → [■] DNS → [▶] Fingerprint → [○] ...   │
└──────────────────────────────────────────────────┘
```

Graph View

ASCII DAG visualization of the full pipeline. Nodes are color-coded:

Color	Meaning
Muted/gray	No items have reached this node yet
Blue	Node has processed items but currently has nothing pending (idle, all upstream subtree processed)
Green	Node has completed all processing
Yellow/flashing	Node is currently processing a batch
Red	Node encountered an error

Navigation: An invisible scrollable list overlays the graph. Scrolling through the list highlights corresponding nodes. Pressing Enter on a highlighted node shows detailed info:

- Items processed / in queue / errors

- Running processes
- Rate limit allocation
- Links to jump to relevant list views (domains, URLs, processes)

Domain View

Sortable, filterable table:

```
┌──────────────────────────────────────────────────────┐
│ Domains (198)                    [Filter: live=true]  │
├──────────────────────────┬──────┬──────┬──────────────┤
│ Name                     │ Live │ IP   │ Source       │
├──────────────────────────┼──────┼──────┼──────────────┤
│ app.target.com           │  ✓   │ 1.2. │ subfinder    │
│ api.target.com           │  ✓   │ 1.2. │ chaos        │
│ staging.target.com       │  ✓   │ 3.4. │ puredns      │
│ old.target.com           │  X   │  -   │ alterx       │
└──────────────────────────┴──────┴──────┴──────────────┘
[/] Search  [f] Filter  [s] Sort  [Enter] Details  [q] Back
```

Pressing Enter on a domain shows detail view: techstack, ports, URLs, DNS records, findings.

URL View

Similar table layout with columns: URL, status code, content type, flags, source. Filterable by status code, flag type, content type.

Process View

Shows running and recent processes:

```
┌──────────────────────────────────────────────────────┐
│ Processes                                            │
├──────────────┬──────────┬──────────┬──────────┬──────┤
│ Tool         │ PID      │ Node     │ Runtime  │ Status │
├──────────────┼──────────┼──────────┼──────────┼──────┤
│ nuclei       │ 12345    │ nuc-url  │ 2m 34s   │ running ▶ │
│ katana       │ 12340    │ crawl    │ 1m 12s   │ running ▶ │
│ subfinder    │ 12330    │ p-enum   │ 45s      │ done ✓    │
└──────────────┴──────────┴──────────┴──────────┴──────┘
```

Pressing Enter on a running process shows live stdout/stderr tail.

Record View

DNS records table: domain, record type, value, TTL.

Findings View

Findings table with severity highlighting. Columns: severity, title, scanner, domain/URL. Sortable by severity.

Scope Page

Form for adding/editing/removing scope rules. Shows current rules with match counts (how many items were filtered by each rule).

14.3 Global Keybindings

Key	Action
`1`-`7`	Jump to page (dashboard, graph, domains, URLs, processes, records, findings)
`?`	Help overlay
`/`	Search/filter current view
`q`	Back / quit (with confirmation)
`p`	Pause/resume pipeline
`o`	Toggle overdrive mode
`e`	Export menu
`Tab`	Cycle focus areas

15. Export

15.1 Artifact Export

Generates individual text files for use with other tools:

```
export/
├── subdomains.txt          # All discovered subdomains (one per line)
├── live-subdomains.txt     # Only live subdomains
├── urls.txt                # All discovered URLs
├── live-urls.txt           # URLs with 200 status
├── ips.txt                 # All IPs
├── ports.txt               # IP:port pairs
├── params/
│   ├── idor.txt            # URLs with id/user params
│   ├── ssrf.txt            # URLs matching ssrf patterns
│   ├── xss.txt             # URLs matching xss patterns
│   ├── redirect.txt        # URLs matching redirect patterns
│   ├── lfi.txt             # URLs matching lfi patterns
│   ├── sqli.txt            # URLs matching sqli patterns
│   └── rce.txt             # URLs matching rce patterns
├── endpoints/
│   ├── api.txt             # API endpoints
│   ├── admin.txt           # Admin/internal paths
│   ├── auth.txt            # Auth endpoints
│   ├── upload.txt          # File upload endpoints
│   └── config.txt          # Config/asset endpoints
├── files/
│   ├── js/                 # Downloaded JavaScript files
│   ├── docs/               # Downloaded documents
│   ├── text/               # Downloaded text/config files
│   └── backups/            # Downloaded backup files
└── findings/
    ├── critical.txt
    ├── high.txt
    ├── medium.txt
    └── low.txt
```

15.2 Obsidian Canvas Export

Generates an Obsidian `.canvas` file (JSON format) that visualizes the recon results as a node graph:

- **Center node**: Target domain
- **First ring**: Subdomains (color-coded by whether they have findings)
- **Edges**: Connect subdomains to their URLs, IPs, findings
- **Groups**: Cluster by techstack, severity, attack surface category
- **Cards**: Findings with severity color and summary

The canvas format is a JSON file with `nodes` (cards, groups, files) and `edges` (arrows between nodes).

15.3 Markdown Report

Generates a structured recon report:

```
# Recon Report: Target Corp
**Date**: 2026-03-30
**Pipeline**: default-recon
**Duration**: 47m 23s

## Summary
- **Subdomains discovered**: 198 (142 live)
- **URLs discovered**: 1,847 (1,203 live)
- **Open ports**: 312 across 67 IPs
- **Findings**: 23 (2 critical, 5 high, 12 medium, 4 low)

## Critical Findings
### CVE-2024-1234 — Remote Code Execution on app.target.com
- **Scanner**: nuclei
- **URL**: https://app.target.com/api/v2/upload
- **Evidence**: ...

## High Findings
...

## Attack Surface Overview
### API Endpoints (47 discovered)
...

### Admin Paths (12 discovered)
...

## Technology Stack
| Domain | Technologies |
|--|-|
| app.target.com | Nginx, React, Node.js |
| api.target.com | Nginx, Express, PostgreSQL |

## Recommendations
1. Focus on app.target.com — highest finding density
2. Investigate API endpoints at api.target.com/v2/
3. Test file upload functionality at ...
4. ...
```

16. AI Integration

16.1 Design Principle

AI is an **extension**, not a core feature. Paravizor is fully functional without any AI configuration. AI enhances the experience for users who opt in.

16.2 Provider Interface

```go
type LLMProvider interface {
    Chat(ctx context.Context, messages []Message) (string, error)
    Stream(ctx context.Context, messages []Message) (<-chan string, error)
    Name() string
    Model() string
}
```

Implementations: OpenAI API, Anthropic API, Ollama (local).

16.3 Features

Pipeline Suggestions

After fingerprinting completes, the AI can analyze the techstack and suggest:

- Additional scanning templates to run
- Known vulnerability patterns for detected technologies
- Priority ordering for manual testing

Requires user consent: AI suggestions are presented, user approves/rejects before execution.

Finding Explanation

In the Findings view, users can ask the AI to explain:

- What the vulnerability means
- Potential impact
- Suggested exploitation path
- Recommended fix

Chat View

A TUI page where users can chat with the AI about the recon results. The AI has context of:

- All discovered domains, URLs, techstack
- All findings
- Pipeline execution history

Users can ask questions like "Which subdomains look most interesting?" or "Explain the tech stack of api.target.com."

Pipeline Modification

The AI can suggest modifications to the pipeline (add a node, change routing, add custom grep patterns). Changes are previewed and require explicit user approval before being applied.

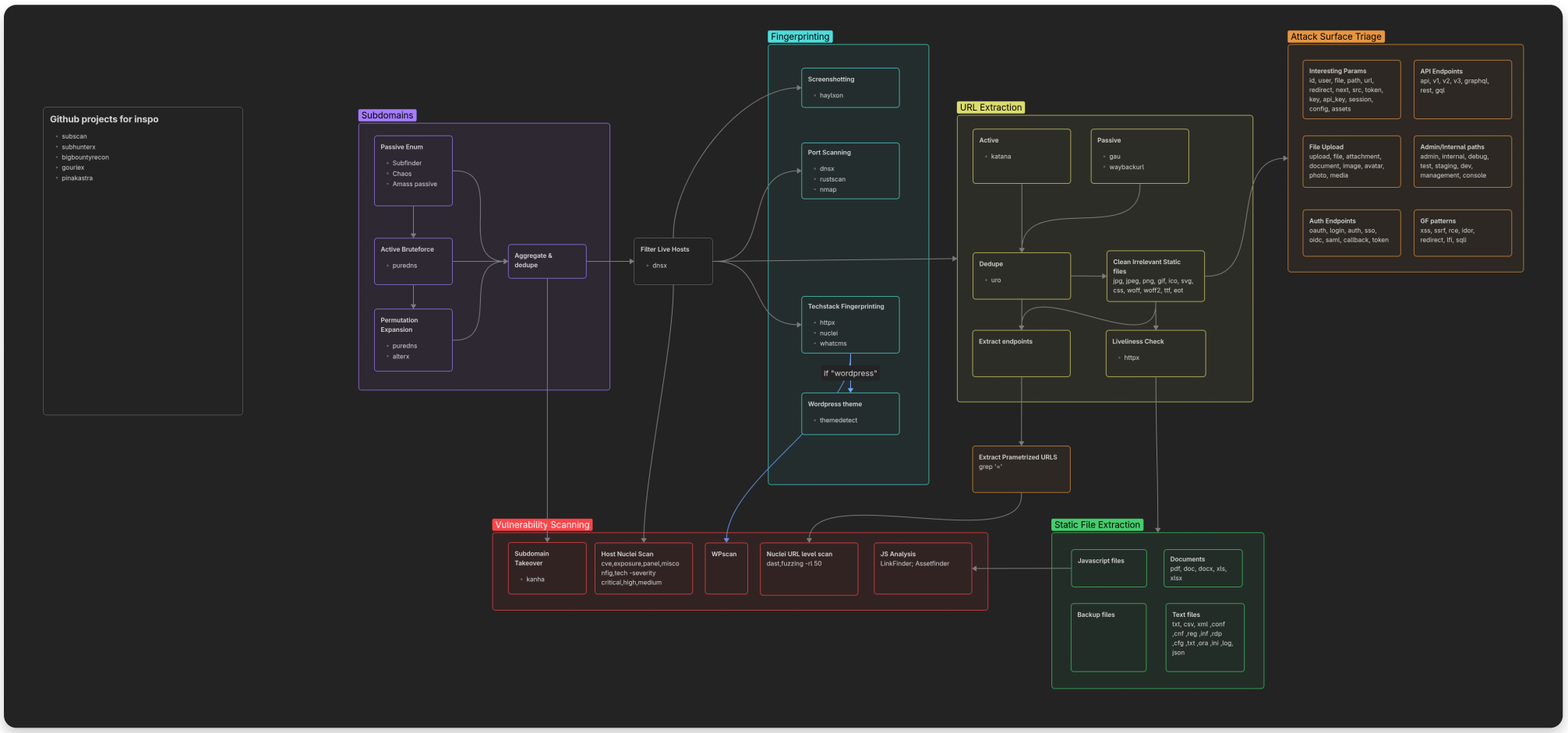
16.4 Consent Model

```
ai:
  consent_mode: always_ask    # always_ask | auto_approve | disabled
```

- **always_ask**: Every AI-initiated action requires user confirmation via TUI modal.
- **auto_approve**: AI suggestions under a certain risk level are auto-applied (read-only actions only, never destructive actions).
- **disabled**: AI features are hidden from the UI.

17. Default Pipeline

The built-in default pipeline should implements the full recon workflow from the original spec. A higher res image will be provided.



18. Collaboration

Now we need to specify how we'll work on this

18.1 Code Specification

- The Go code should be well structured and follow best practices
- Dependencies should be decoupled as much as possible (don't import sqlit in the tui module for example)
- Modules should do one job and one job only
- Functions that are used across multiple modules should be defined in the `utils` module
- Type definitions should be defined in a `types.go` file in each module
- Comments should explain ambiguous functions and decisions
- Each functionality should have an accompanying test file to test its functionality and edge cases. **THIS IS MANDATORY**
- Documentation should be written in parallel with functions and structures. It should be thorough and explain how everything works *(see the `docs/` directory)*

18.2 Phases

Work will be split into two phases. The first one will have 4 main functionalities, and the second will have two. The ones working on the **UX** and **Engine** modules will only work on those since they're the most complex. The ones working on the **DB** and **Config** should choose 1 other functionality from the second phase.

- **Phase 1**: Core (MVP)
  - **DB**: You'll be responsible for making the per project and global store, queries, handlers, and definitions. This includes syncing and maintaining data consistency
  - **Engine**: You'll be responsible for making the core logic for the tool. Building the dag, routing between nodes, filtering, and communicating the events through the bus
  - **Config**: You'll be responsible for handling and parsing the config, pipeline and tool definition. This includes tool validation, skipping invalid nodes, loading and merging configs.
  - **UX**: This includes building the CLI and TUI aswell as the event bus and everything that comes with user interactivity.
- **Phase 2:** Polish
  - **Rate Limit**: Building the dynamic rate limiter and credit system
  - **AI**: Integrating the AI into the app

Previous work has been conducted on this project. It looks sound but it's very unstable. You can reference it from here for your work (old repo)

Note that that old repo has **unsafe and untested code!** Don't copy paste from it blindly, but you can check it to get additional insight on how stuff should theoretically work

Appendix: Glossary

Term	Definition
Node	A processing unit in the pipeline DAG. Wraps one or more tools.
Stage	A logical grouping of nodes (for visualization and ordering).
Batch	A group of items processed together by a node.
Route	A directed edge in the DAG, optionally with a condition.
Item	A unit of data flowing through the pipeline (domain, URL, IP, etc.).
Credit	A unit of rate-limit capacity (1 credit = 1 request/second).
Credit pool	The global per-target rate-limit budget, distributed among active nodes.
Burst reserve	A portion of the credit pool held for newly activating nodes.
Tool definition	A YAML file describing how to execute and parse a tool.
Tool runner	The Go component that executes external tool processes.
Overdrive	Mode that bypasses all rate limiting.
Checkpoint	A saved snapshot of pipeline state for crash recovery.